

Question:	1	2	3	4	Total
Points:	10	8	9	9	36
Score:					

Instructor: Dr. Ali Alsaffar.

(1) Answer the following questions.

- (a) [2 points] In which step of Algorithmic Problem Solving you trace the algorithm's performance for specific inputs? Write the title of this step.

Solution: Prove Algorithm's Correctness

- (b) [2 points] Suppose you are required to write an algorithm to count how many words in a string starts with uppercase. What is the problem type of this algorithm?

Solution: String Processing.

- (c) [1 point] Give a name of an algorithm that implements Exhaustive Search.

Solution:

1. Travelling Salesman.
2. Knapsack Problem.
3. Job Assignment.

- (d) [2 points] Consider the time complexity $T(n) = 64T(n/4) + n^2 + 1$. Use the *Master Theorem* to prove that $T(n) \in \Theta(n^3)$.

Solution: $a = 64, b = 4, d = 2$ and $a = 64 > 4^2 = 16$
 $\therefore T(n) \in \Theta(n^{\log_4 64}) = \Theta(n^{\log_4 4^3}) = \Theta(n^3)$

- (e) [3 points] Fill in the table below for each *iteration* of the **Selection Sort** algorithm when it is applied on the array: $A = [11, 17, 15, 10, 12]$.

	Values of A				
Iteration	11	17	15	10	12
1	10	17	15	11	12
2	10	11	15	17	12
3	10	11	12	17	15
4	10	11	12	15	17

- (2) [8 points] Suppose you are given an array, $A[0..n-1]$, of n positive integers. Write a brute-force and *linear* (with $O(n)$ time complexity) algorithm for removing all the even numbers from A and keeping the odd numbers in A with their relative original order.

	Before								After					
$A \rightarrow$	2	5	9	12	17	25	26	31	$A \rightarrow$	5	9	17	25	31

Solution:

In-Place Algorithm

```

1: Algorithm REMOVEEVEN( $A[0..n-1]$ )
2:    $k \leftarrow 0$ 
3:   for  $i \leftarrow 0$  to  $n-1$  do
4:     if  $A[i]$  is odd then
5:        $A[k] \leftarrow A[i]$ 
6:        $k \leftarrow k+1$ 
7:    $n \leftarrow k$ 

```

None In-Place Algorithm

```

1: Algorithm REMOVEEVEN( $A[0..n-1]$ )
2:   DECLARE  $B[0..n-1]$ 
3:    $k \leftarrow 0$ 
4:   for  $i \leftarrow 0$  to  $n-1$  do
5:     if  $A[i]$  is odd then
6:        $B[k] \leftarrow A[i]$ 
7:        $k \leftarrow k+1$ 
8:   for  $i \leftarrow 0$  to  $k-1$  do
9:      $A[i] \leftarrow B[i]$ 
10:   $n \leftarrow k$ 

```

- (3) Consider the following program segment.

```

1:  $A \leftarrow 2$ 
2: for  $i \leftarrow 1$  to  $2n$  do
3:   if  $i$  is even then
4:     for  $k \leftarrow 1$  to  $i$  do
5:        $A \leftarrow A+2$ 
6:   else
7:      $A \leftarrow A*3$ 

```

- (a) [2 points] What is the basic operation? At which line?

Solution: The addition at line 5

- (b) [6 points] Find number of additions $A(n)$, at line 5. Use summations to express your answer.

Solution:

$$\begin{aligned}
 A(n) &= \sum_{i=2,4,\dots,2q} \sum_{k=1}^i 1 = \sum_{i=2,4,\dots,2q} i = 2 + 4 + \dots + 2q \\
 &= 2(1 + 2 + \dots + q) \\
 &= 2 \cdot \frac{q(q+1)}{2} = q^2 + q \quad (1)
 \end{aligned}$$

The last iteration of the **FOR** loop is $2q \leq 2n$, which is $q \leq n$.

Estimated answer: Let $q = n$, in (1), $A(n) = q^2 + q = n^2 + n$

Exact answer:

$$\begin{aligned}
 q &\leq n < q + 1 \\
 q &\leq n < q + 1 \\
 \therefore \lfloor n \rfloor &= q
 \end{aligned}$$

In (1), $A(n) = q^2 + q = \lfloor n \rfloor^2 + \lfloor n \rfloor$.

- (c) [1 point] What is the time efficiency of the algorithm (order of growth)?

Solution: $A(n) = n^2 + 2 \in \Theta(n^2)$.

- (4) Consider the following recursive algorithm.

```

1: Algorithm F(n)
2:   if (n > 0) then
3:     s ← 1
4:     for i ← 1 to 5 do
5:       s ← s × F(n − 1)
6:   PRINT(s)

```

- (a) [2 points] Write the recurrence relation for the time complexity of the algorithm $T(n)$. What is the basic operation?

Solution: The basic operation is the multiplication at line 5.
The time complexity recurrence $T(n)$:

$$T(n) = \begin{cases} 0 & n = 0 \\ 0 + 5T(n-1) + 5 & n > 0 \end{cases}$$

DIR(0) = 0, PRE(*n*) = no multiplications, POST(*n*) = 5 multiplications

- (b) [6 points] Find the solution of $T(n)$ and its order of growth.

Solution:

$$\begin{aligned}
T(n) &= 5T(n-1) + 5 && \leftarrow \text{step 1} \\
&= 5^2T(n-1) + 5^2 + 5 && \leftarrow \text{step 2} \\
&= 5^3T(n-1) + 5^3 + 5^2 + 5 && \leftarrow \text{step 3} \\
&\vdots \\
&= 5^kT(n-1) + \sum_{j=1}^k 5^j && \leftarrow \text{step } k \\
&= 5^kT(n-1) + 5 \sum_{j=0}^{k-1} 5^j = 5^kT(n-1) + \left(\frac{5}{4}\right)(5^k - 1)
\end{aligned}$$

Until $T(n-k) = T(0)$, $n-k=0 \implies k=n$, and

$$\therefore T(n) = 5^nT(0) + \left(\frac{5}{4}\right)(5^n - 1) = \left(\frac{5}{4}\right)(5^n - 1).$$

Growth rate: $T(n) \in \Theta(5^n)$.

- (c) [1 point] In short, why the algorithm is not considered a *divide-and-conquer* algorithm?

Solution: Because the algorithm is not dividing n by a constant, it is decreasing n by constant.